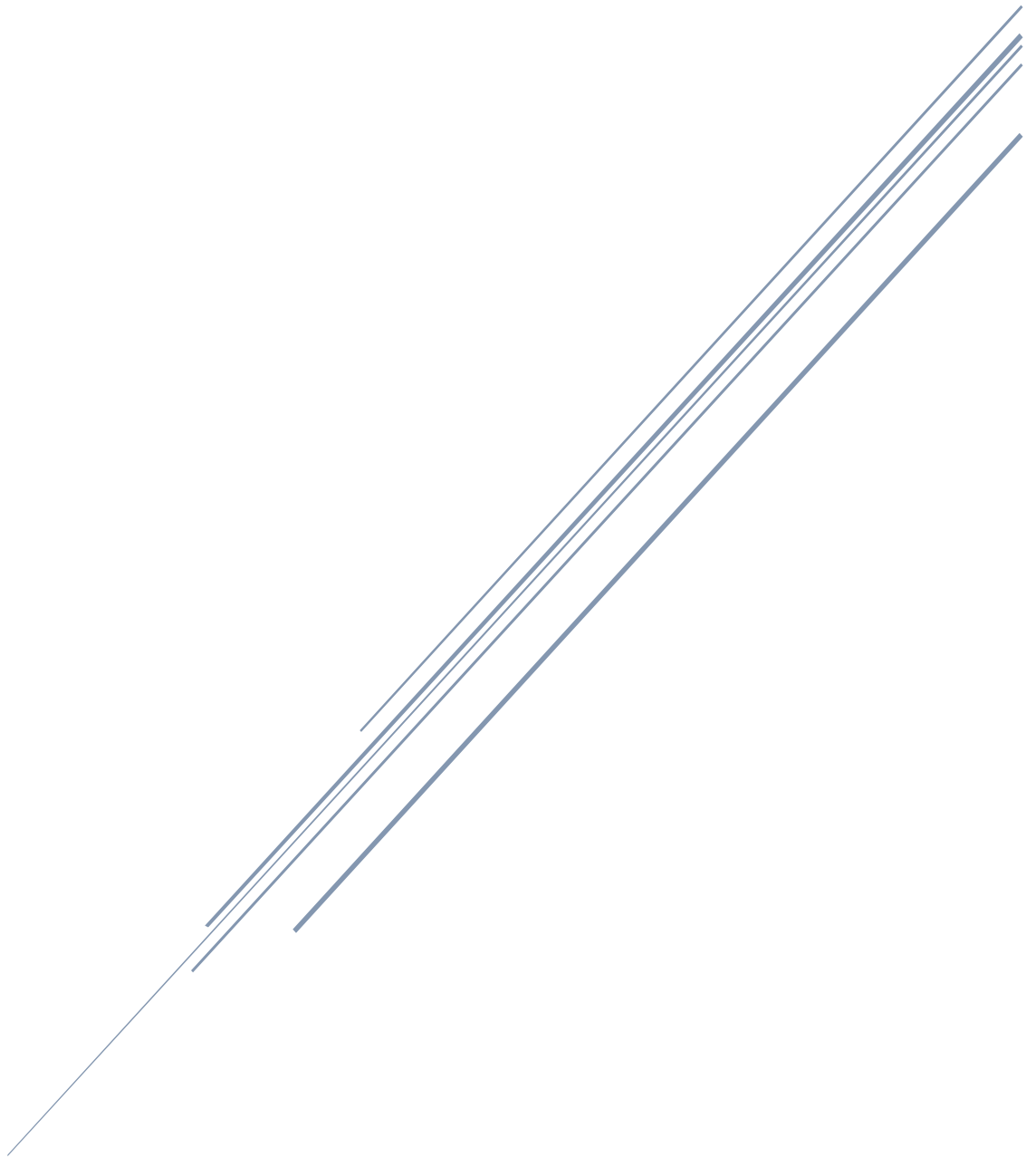


Service Oriented Computing

Individual Assignment



Proposed By :Mr. Gihan Herath
Completed By :M.S.P Kavishka Manohara Menikbowa

Assignment Cover Sheet

Qualification		Module Number and Title	
Higher Diploma in Computing and Software Engineering		CSE5013/ Service Oriented Computing	
Student Name & No.		Assessor	
Menikbowe Sakalawalli Patabadigera Kavishka Manohara Menikbowa KD/HDCSE/CMU/77/50		Mr. Gihan Herath	
Hand out date		Submission Date	
2026-02-13		2026-04-10	
WRIT1- Coursework	Duration/Length of Assessment Type	Weighting of Assessment	
	End of the module	100%	

Learner declaration
I, Menikbowe Sakalawalli Patabadigera Kavishka Manohara Menikbowa (KD/HDCSE/CMU/77/50), certify that the work submitted for this assignment is my own and research sources are fully acknowledged.

Marks Awarded			
First assessor			
IV marks			
Agreed grade			
Signature of the assessor		Date	

Feedback Form

INTERNATIONAL COLLEGE OF BUSINESS & TECHNOLOGY

Module: CSE5013/ Service Oriented Computing

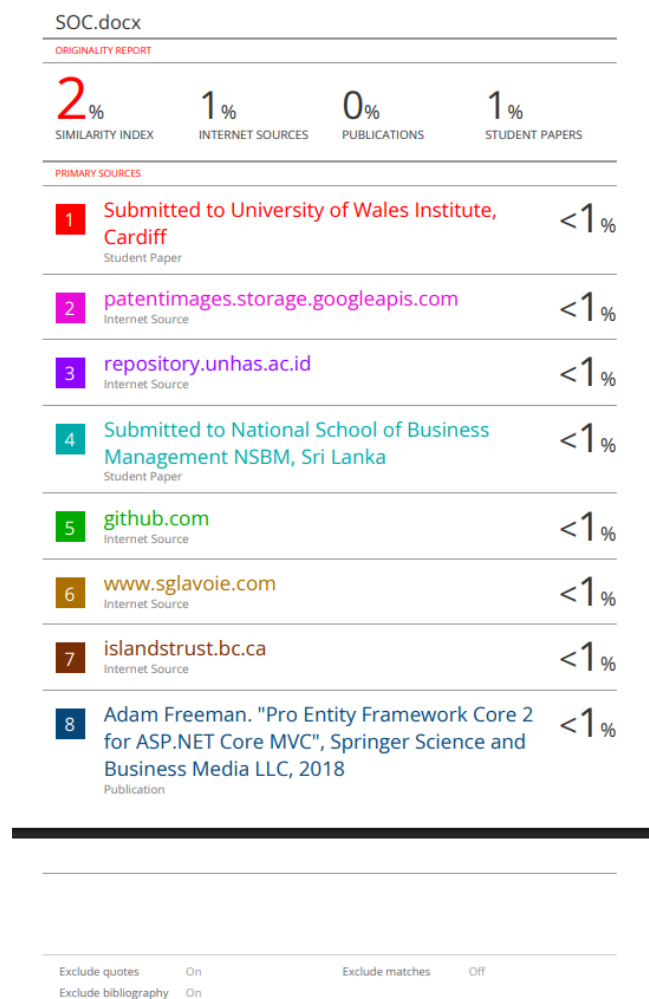
Student ID: KD/HDCSE/CMU/77/50

Assignment: Writ1

Marks Allocated	Task 1 10	Task 2 20	Task 3 15	Task 4 25	Task 5 20	Task 6 10	Total marks 100
Marks Awarded							

Task	Feedback	
1		
2		
3		
4		
5		
6		
General Comment		
Assessor Name	Signature	Date

Turnitin Report



Acknowledgement

My sincere thanks are due to all people who helped make the project completed successfully.

I gratefully acknowledge my lecturer Mr. Gihan Herath for her valuable guidance and continuous encouragement throughout this journey. I want to express my gratitude to my classmates and co-workers for their supportive feedback as well as team spirit.

Also, I recognize the help and services that the ICBT Campus facilities provide.

Endless thanks to my family alongside my friends for their ceaseless motivation and support.

Thanks a lot to everyone who has helped me and supported me along the way.

Table of Contents

1. Introduction.....	6
1.1 Project Overview: Kandy Municipal Council (KMC) Event Platform	6
1.2 Problem Statement & Objectives.....	7
Problem Statement.....	7
Project Objectives	7
2. Architectural Analysis.....	8
2.1 Monolithic Architecture: Overview and Limitations.....	8
2.2 Service-Oriented Architecture (SOA): Overview and Benefits.....	9
2.3 Comparative Analysis: Monolithic vs. SOA.....	10
2.4 Justification of SOA for KMC (Maintainability & Scalability).....	11
3. System Design & SOC Principles.....	12
3.1 Service Identification and Design.....	12
3.2 Key SOC Principles Applied	14
3.2.1 Loose Coupling & Service Autonomy	14
3.2.2 Service Reusability & Abstraction.....	16
3.3 Design Diagrams.....	17
3.3.1 High-Level Architecture Diagram	17
3.3.2 Use Case Diagrams	17
3.3.3 Class Diagram.....	20
3.3.4 Technical Sequence Diagram.....	21
4. Implementation Details.....	22
4.1 Development Environment & Technology Stack	22
4.2 Core Service Implementation (KmcEventApi).....	23
4.3 Service Consumers Development	24
4.3.1 KmcOrganizerPortal Implementation	24
4.3.2 KmcPublicWeb (Resident/Public Interface)	26
4.4 Final System Interfaces (Service Consumers in Action)	27
4.4.1 KmcOrganizerPortal - Dashboard View	27
4.4.2 KmcPublicWeb - Resident View.....	28
5. Testing & Validation	29
5.1 API Testing using Postman	29
1. GET Request (Data Retrieval)	29

2 POST Request Validation (Resource Creation)	30
5.2 Validation of Business Rules	31
5.3 Error Handling and Status Codes.....	32
6. Critical Evaluation	33
6.1 Benefits of Using SOC for KMC	33
6.2 Challenges Faced During Implementation.....	34
6.3 Future Scalability (Adding Payment or Notification Services)	35
7. Conclusion & Self-Reflection.....	36
8. References.....	37
9. Appendices (Source Code Links).....	37

List Of Figure

Figure 1 Service Identification & System Architecture	12
Figure 2 REST API Request-Response Cycle	13
Figure 3 SOA Interaction Model (KMC Event Platform).....	14
Figure 4 Loose Coupling in KMC Platform	15
Figure 5 Service Autonomy Model.....	15
Figure 6 Service Reusability Model.....	16
Figure 7 Service Abstraction (Black Box Concept).....	17
Figure 8 High-Level Architecture Diagram (KMC Platform)	17
Figure 9 Client (Organizer Portal) Use Case	18
Figure 10 API (Core Service) Use Case.....	19
Figure 11 Web (Public Web) Use Case	19
Figure 12 System Class Diagram (KMC Event Platform).....	20
Figure 13 Technical Sequence Diagram (Data Retrieval).....	21
Figure 14 Solution Explorer Screenshot showing project structure.....	23
Figure 15 EventsController Code Implementation	24
Figure 16 KmcOrganizerPortal API Consumption Code (auth.js).....	25
Figure 17 KmcPublicWeb API Integration (Swagger & Network Tab).....	26
Figure 18 Organizer Portal Dashboard - A screenshot of the web application showing the event management table/dashboard.....	27
Figure 19 KMC Public Website - A screenshot of the public-facing website showing the latest event listings.....	28
Figure 20 API GET Request Validation (Swagger)	29
Figure 21 API POST Request Validation (Swagger)	30
Figure 22 API PUT Request Validation (Swagger).....	31
Figure 23 Error Handling Validation (404 Not Found).....	32

1. Introduction

1.1 Project Overview: Kandy Municipal Council (KMC) Event Platform

The Kandy Municipal Council (KMC) serves as the local government authority that manages Kandy, which is a city known for its historical and cultural significance and active community life. The KMC currently struggles to efficiently promote city events, which results in decreased community participation and ineffective information distribution.

The project implements a Service-Oriented Computing (SOC) based solution to overcome existing difficulties. The "KMC Event Platform" serves as a digital platform which enables event organizers and city residents to communicate with each other through one central online platform.

The platform utilizes Service-Oriented Architecture (SOA) which enables users to access system functions through a central API while keeping all system components separate from each other. The system contains three primary components which developers built with current .NET technologies.

- KmcEventApi (Service Provider): A robust backend service developed with .NET 10.0 that handles the core business logic, including event creation, management, and data persistence.
- KmcOrganizerPortal (Service Consumer): A dedicated web portal for event organizers to register and manage their specific events.
- KmcPublicWeb (Service Consumer): A public-facing web interface that allows residents to discover, search, and view upcoming events in the city.

The KMC implements an SOC approach which enables platform maintenance while supporting system growth and future service integration through minimal impacts on existing system elements.

1.2 Problem Statement & Objectives

Problem Statement

The Kandy Municipal Council (KMC) needs a central automated system to manage and promote city-wide events. The current manual and partial systems create multiple operational difficulties.

- **Information Silos** The scattered event details make it hard for residents to find all information in one location.
- **Inefficient Management** The organizers need a simple process to change event information but they lack this capability which results in incorrect data.
- **Scalability Issues** A non-modular system becomes harder to manage and extend when the city event count increases.
- **Lack of Integration** Different applications (Web, Mobile, Portals) need an API to exchange event data which requires a standard method of communication.

Project Objectives

Project Objectives The project aims to create a Service-Oriented Computing (SOC) system which updates KMC's event management functions. The project focuses on these specific goals.

- **Service-Oriented Design** The core API (Service Provider) will function as the event management system because it handles all event-related functions.
- **Stakeholder Integration** The project will create specific client applications (Service Consumers) which serve both event organizers (Portal) and the general public (Web).
- **Enhanced Maintainability** The application of SOC principles including Loose Coupling and Abstraction will enable KMC to replace system components without disrupting the complete system.
- **Data Consistency** The system provides one reliable source to access all event-related data through its RESTful API.
- **Security & Ownership** Business rules will let only authorized creators to control their events which protects data integrity.

2. Architectural Analysis

2.1 Monolithic Architecture: Overview and Limitations

Overview

Monolithic architecture represents traditional software development through its approach which constructs complete applications as single, unbreakable entities. The model requires all components to function together through tight integration with a shared codebase which encompasses the user interface and business logic and data access layers. The Kandy Municipal Council (KMC) event system needs a monolithic design approach because it requires all components from the Organizer Portal and Public Web and event management logic to combine into one comprehensive executable or server.

The KMC organization faces multiple challenges which arise from its use of monolithic architecture:

- The system becomes fragile because all components depend on each other. The public search feature would break because of a small change which affects the event registration logic.
- Deployment Challenges: Even for a minor update to a single page in the Organizer Portal, the entire KMC system must be recompiled and redeployed. This causes extended downtime periods which lead to increased operational risks.
- Scalability Constraints: The monolithic architecture prevents independent scaling of application components. The whole application needs to scale up during peak Kandy festival period because Public Web experiences excessive user traffic which results in inefficient resource distribution problems.
- The system requires developers to implement all features using one specific technology stack. The KMC organization faces an integration challenge because it needs to implement new features through framework usage with its current monolith structure.
- The codebase becomes harder to maintain because the city services add additional content. The code now functions as a "Big Ball of Mud," which prevents new developers from understanding its structure and maintaining their work.

2.2 Service-Oriented Architecture (SOA): Overview and Benefits

Overview

Service-Oriented Architecture (SOA) establishes an architectural framework which enables software functions to be delivered as separate "services" that interact through network communication. The monolithic method connects all application components into a single system while SOA enables systems to function through independent services which deliver particular business operations. The Kandy Municipal Council (KMC) platform uses KmcEventApi to protect its main business rules which the two user interfaces (Organizer Portal and Public Web) use as their method to access this API.

Benefits of SOA for the KMC Project:

- The project enables loose coupling between frontend applications which function as Consumers and the backend API which acts as the Provider. The KmcPublicWeb UI can be altered independently from the KmcEventApi because the two systems function as independent entities.
- The KmcEventApi operates as a reusable resource. The same event-fetching service is used by both the KmcOrganizerPortal (for management) and the KmcPublicWeb (for public viewing). This solution prevents developers from having to write identical code multiple times.
- The KMC system achieves better scalability through its ability to independently expand both the KmcPublicWeb and the KmcEventApi base system which provides capacity for public access during peak event viewing times.
- SOA establishes a framework which enables different platforms to exchange information through standard protocols (like HTTP/REST) and data formats (like JSON). This allows KMC to create a future mobile application which will use the existing API.
- The maintenance process becomes simpler because separate services function as distinct projects which enable teams to test and apply updates throughout systems. Developers can focus on one service at a time without worrying about the entire system's complexity.

2.3 Comparative Analysis: Monolithic vs. SOA

The Kandy Municipal Council project shows benefits from service-oriented architecture because its performance needs direct comparison with the monolithic system. The following analysis highlights the trade-offs between the two architectures in terms of development, deployment, and management.

Detailed Comparison:

- **Development Complexity:** In a Monolith, developers must understand the entire codebase even to make a small change. The KmcEventApi functions as an independent service in SOA, which enables developers to create business logic without needing to handle Frontend UI code.
- **Deployment Frequency:** A Monolithic KMC app would require a full system restart for any update. The system allows us to perform KmcPublicWeb updates, which include theme and layout changes, while the API continues to operate.
- **Data Management:** In a Monolith, all modules usually share a single database with high dependency. SOA enables database sharing between systems, but users must access data through the service layer API to maintain data integrity.
- **The expensive process of scaling Monoliths** requires users to upgrade to advanced hardware. The cost-effective and efficient design of SOA enables cities to implement city-scale applications through its ability to scale by adding more service instances.

Comparison Summary Table:

Feature	Monolithic Architecture	Service-Oriented Architecture (SOA)
Coupling	Tightly Coupled	Loosely Coupled
Deployment	Single deployment unit	Independent service deployment
Scaling	Scale the entire application	Scale individual services
Fault Isolation	Low (One crash affects all)	High (Failure is isolated to a service)
Agility	Slow (Hard to change/update)	Fast (Modular updates)

2.4 Justification of SOA for KMC (Maintainability & Scalability)

The Kandy Municipal Council (KMC) event platform should use Service-Oriented Architecture (SOA) according to the results of the comparative analysis. The selection proceeds according to its two main software quality attributes which both need to be maintained through the process of software development.

Justification based on Maintainability:

- The KMC platform will serve as a permanent digital resource for its users. The codebase growth in a monolithic system leads to maintenance challenges which escalate at an increasing rate. The SOA approach allows maintenance to proceed with these benefits.
- Modular Design: Developers require API modifications to create new event registration rules because they can develop the KmcEventApi core logic separately from KmcOrganizerPortal and KmcPublicWeb user interfaces.
- Simplified Debugging: The independent operation of services enables developers to find errors with greater efficiency. Developers can test the API as a separate component to identify whether the public web event viewing problems originate from data service issues or client-side rendering complications.
- The KMC system uses SOA which allows them to release KmcOrganizerPortal updates based on organizer feedback while maintaining public-facing website functionality.

Justification based on Scalability:

The platform experiences variable usage patterns because Kandy serves as a center for major cultural festivals which occur throughout the year.

- Horizontal Scaling: During the Esala Perahera peak traffic time, KMC needs to increase KmcPublicWeb and KmcEventApi capacity through horizontal scaling techniques which involve adding more instances to support thousands of simultaneous users. The application needed to be expanded from its monolithic structure because scaling the entire system would consume server capacity for the low-usage Organizer Portal.
- Future-Proofing: SOA allows organizations to introduce new services into their systems. The KMC plans to create two new services which will function as "Ticket Booking Service" and "SMS Notification Service" that will connect through the API to the current system without needing existing system rewrites.

The KMC can implement SOA as a flexible and strong yet budget-friendly system which will grow with the city's development while remaining manageable for technical staff.

3. System Design & SOC Principles

3.1 Service Identification and Design

The Service-Oriented Computing (SOC) environment starts with the process of discovering business functions which can be developed into standalone services. The KMC Event Platform system implementation divides system requirements into Core Service and Consumer Applications components.

Service Identification:

The primary service identified is the Event Management Service which operates through its Web API interface KmcEventApi. The service functions as the Service Provider. It contains all data handling functions together with business rules that govern Kandy event management operations.

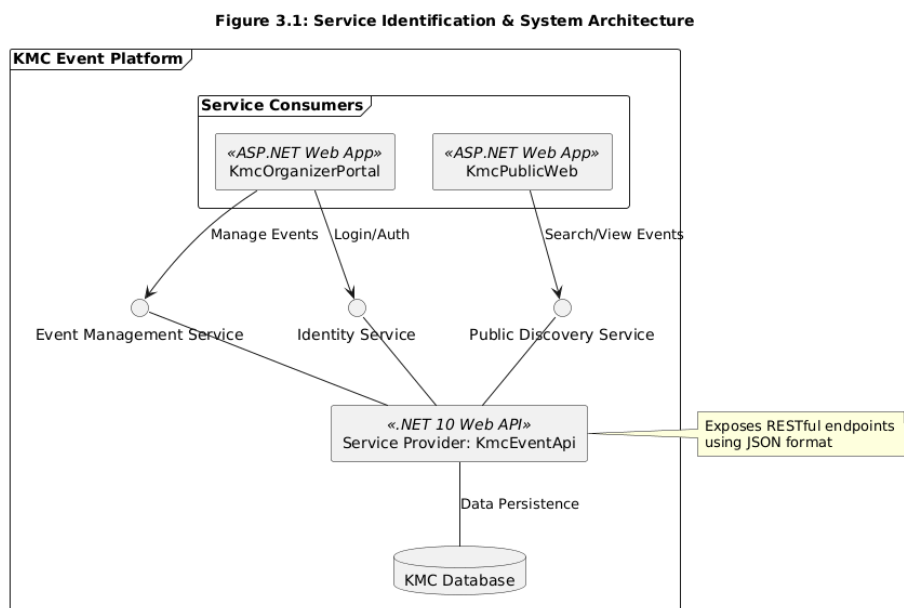


Figure 1 Service Identification & System Architecture

The following core functionalities (services) were identified and mapped to the API:

1. Event Discovery Service (GET): Allows any consumer to fetch a list of all active events or search for specific events.
2. Event Lifecycle Management (POST/PUT/DELETE): Authorized organizers can use this feature to create new events and update existing events and delete events from the system.
3. Identity Service: The system manages user authentication to determine if someone is a general resident or an event organizer. The system manages user authentication to determine if someone is a general resident or an event organizer.

Service Design:

- The service is designed using REST (Representational State Transfer) principles. The service maintains its small size which enables all users to access it through standard HTTP methods on both Web and Mobile platforms.
- Contract Design: The service uses JSON (JavaScript Object Notation) as the data exchange format. The data transmission process between KmcEventApi and its consumers establishes the Service Contract which describes both sending and receiving data procedures.

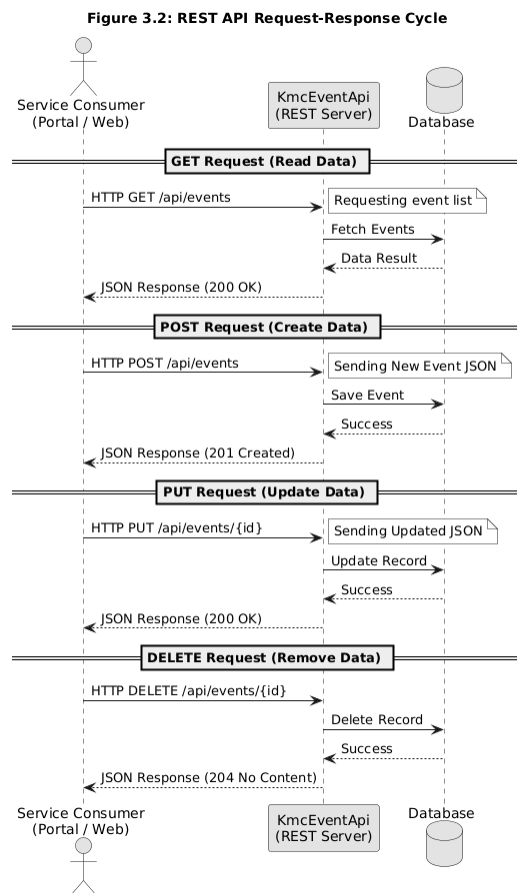


Figure 2 REST API Request-Response Cycle

Endpoint Mapping:

- GET /api/events - Retrieves complete event information from the system.
- POST /api/events - Accepts a new event object to be saved in the database.
- PUT /api/events/{id} - Updates an existing event based on the ID.

Figure 3.3: SOA Interaction Model (KMC Event Platform)

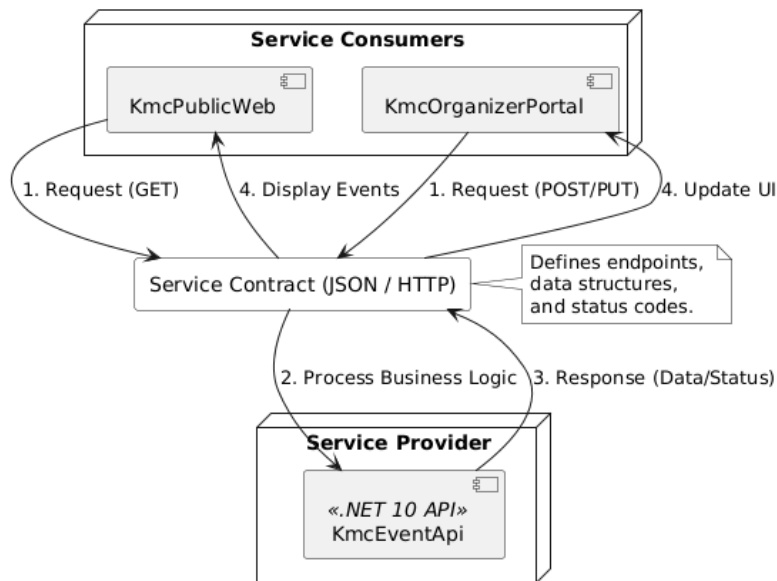


Figure 3 SOA Interaction Model (KMC Event Platform)

The KmcEventApi system functions as an independent unit because its design protects its internal database saving process from visibility to both KmcPublicWeb and KmcOrganizerPortal users. The KmcEventApi system functions as an independent unit because its design protects its internal database saving process from visibility to both KmcPublicWeb and KmcOrganizerPortal users.

3.2 Key SOC Principles Applied

3.2.1 Loose Coupling & Service Autonomy

1. Service Loose Coupling

The SOC framework uses loose coupling as its fundamental principle which enables services to function with decreased need for mutual dependency. The KMC Event Platform establishes client application connections to KmcEventApi which functions as its service provider through dedicated service contract boundaries that utilize REST API technology.

The KmcPublicWeb and KmcOrganizerPortal do not have direct access to the database. The system requires API endpoints together with JSON structure information for its operations. The API internal logic permits changes because developers can modify the system without needing to touch any frontend application components

The system benefits from this feature because it decreases the transmission of disruptions which leads to system malfunctions in other areas.

Figure 3.4: Loose Coupling in KMC Platform

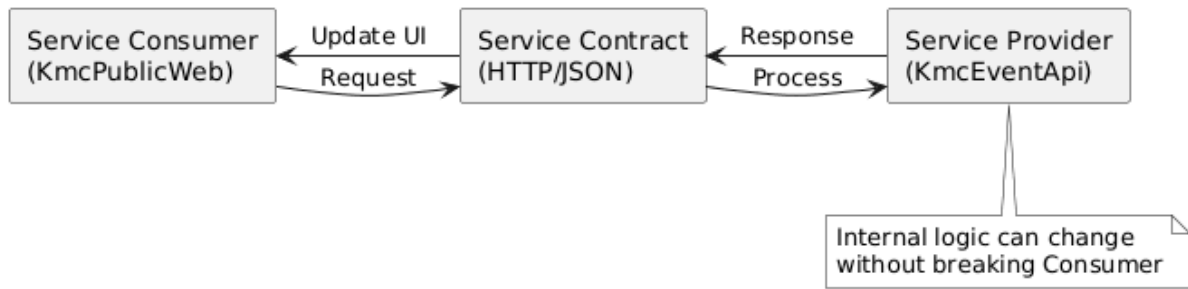


Figure 4 Loose Coupling in KMC Platform

2. Service Autonomy

The service autonomy of a service determines its ability to manage its implementation and operational environment. The KMC system has built autonomous capabilities for all its components.

Application in Project: * Design-time Autonomy: The KmcEventApi development process created its own independent system which functions outside of frontend portals. It has developed its own methods to validate events and protect security.

Runtime Autonomy: Each project (API, Portal, Web) can be hosted on different servers or environments. The API can operate on a powerful Linux system while the Web portal uses its own cloud-based environment.

Benefit: The KmcEventApi and KmcOrganizerPortal maintain their operational status after a KmcPublicWeb crash or high traffic situation because they function as independent systems.

Figure 3.5: Service Autonomy Model

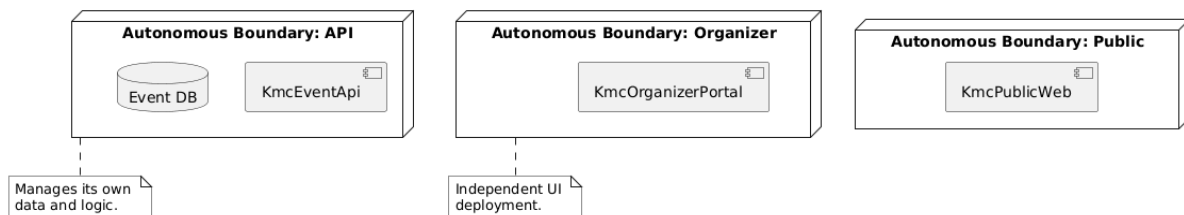


Figure 5 Service Autonomy Model

3.2.2 Service Reusability & Abstraction

1. Service Reusability

The principle of Service Reusability states that service developers should create services which different users can access through multiple usage methods. The KMC Event Platform uses its KmcEventApi system to maintain centralized logic which all interface modules can access for backend operations.

Project Application: The KmcEventApi contains the "Event Retrieval" service which demonstrates its reusable capabilities. The KmcPublicWeb consumes it to show event listings for city residents while KmcOrganizerPortal uses it to display registered events to organizers. The same API service is reused to implement data-fetching logic across all portals instead of creating distinct data-fetching logic for each portal.

The system achieves a "single source of truth" while decreasing code maintenance needs and speeding up client application development through this method.

Figure 3.6: Service Reusability Model

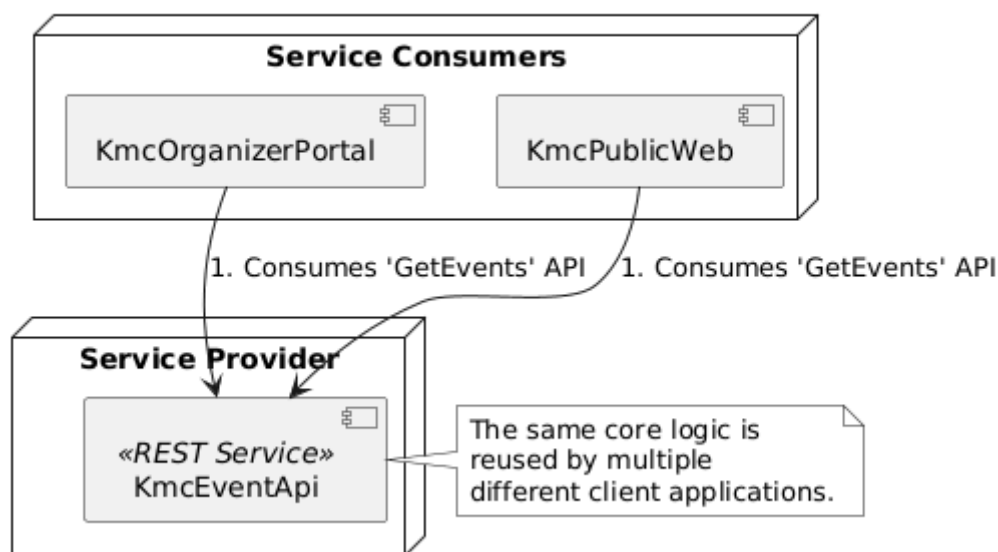


Figure 6 Service Reusability Model

2. Service Abstraction

The system allows users to access services through simplified interface points which hide all technical specifications and operational details of the backend system. The service contract functions as the sole interaction point between the user and the service.

The organizer creates new events through the KmcOrganizerPortal which sends a JSON request to KmcEventApi. The portal (Consumer) has no knowledge of the underlying database schema, the C# validation logic, or the server-side processing. The system only knows which data to transmit and what kind of answer to await.

The system maintains its complete operational capacity through abstraction which lets backend developers make database and internal logic changes without impacting the user experience or front-end development.

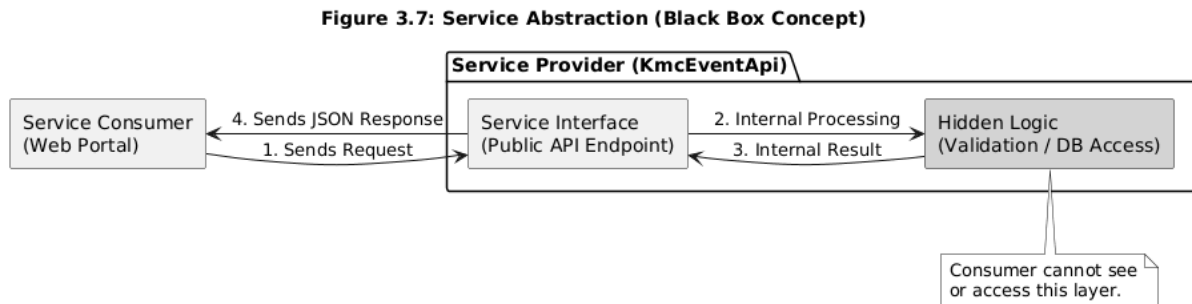


Figure 7 Service Abstraction (Black Box Concept)

3.3 Design Diagrams

The KMC Event Platform architecture and behavioral logic are visually represented through this section which demonstrates the Service-Oriented Computing (SOC) requirements.

3.3.1 High-Level Architecture Diagram

The system implements a decoupled design which allows independent operation between the Service Provider (API) and Service Consumers (Web Apps). This design enables Kandy Municipal Council to conduct individual maintenance and component upgrading for their system.

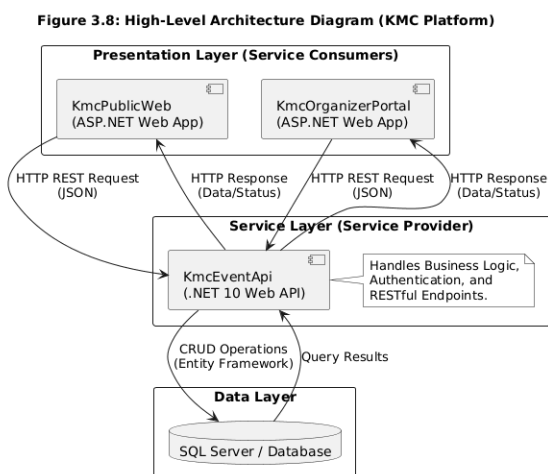


Figure 8 High-Level Architecture Diagram (KMC Platform)

3.3.2 Use Case Diagrams

Three distinct Use Case diagrams have been created to present the functional requirements of the Client Portal, Core API, and Public Web from various user perspectives.

1. Client (Organizer Portal) Use Case

The diagram displays the system functions which event organizers use to handle their upcoming events.

- Primary Actor: Event Organizer
- Core Actions: The user logs into the system to create and edit and delete events.

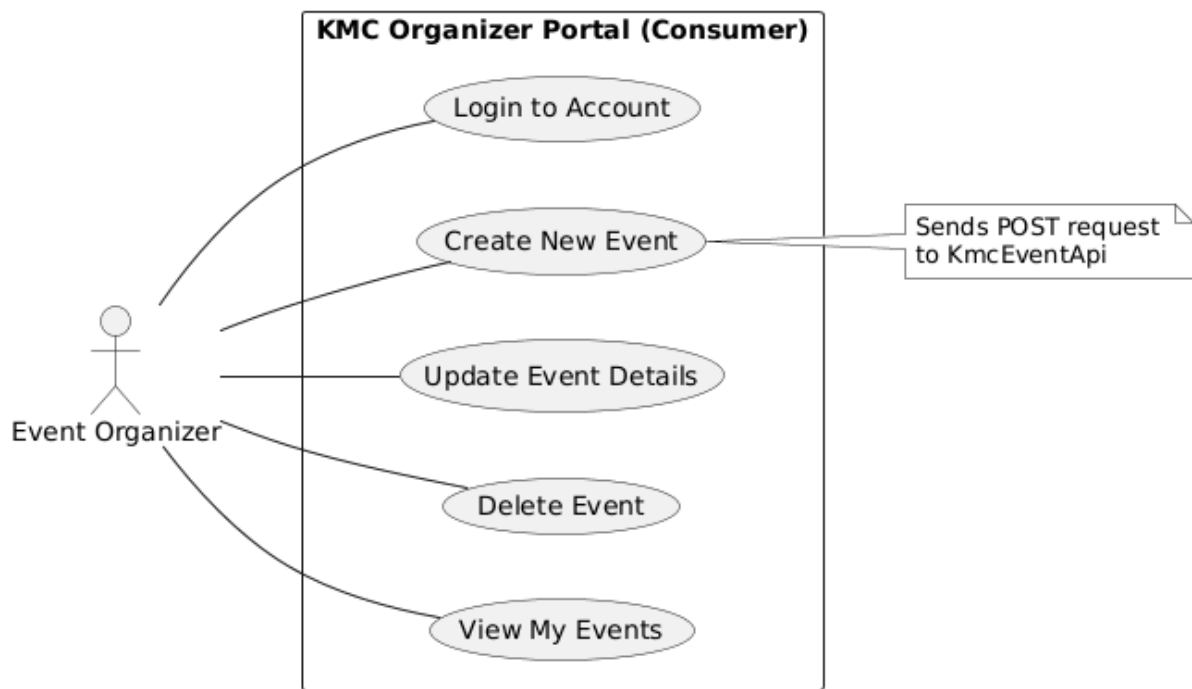


Figure 9 Client (Organizer Portal) Use Case

2. API (Core Service) Use Case

The diagram displays the internal service operations which KmcEventApi uses to run its services. The service system handles requests from all users who have verified their identity.

- Primary Actor: Service Consumer (Internal System)
- Core Actions: The system validates user data through authentication and data validation and record retrieval/storage.

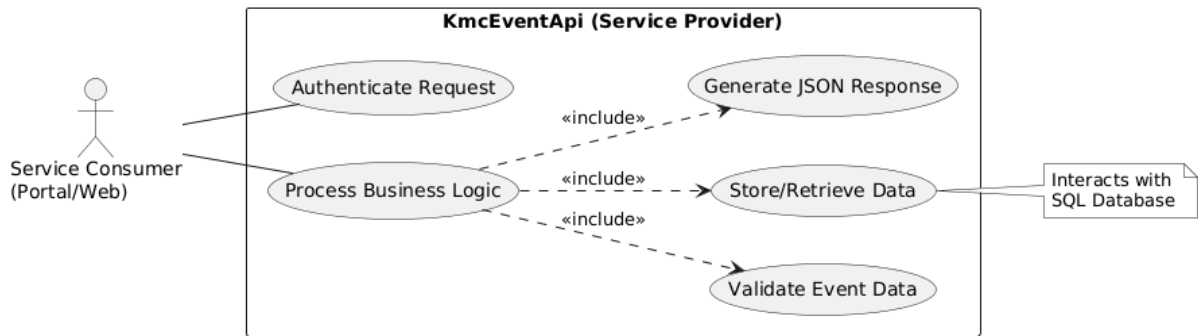


Figure 10 API (Core Service) Use Case

3. Web (Public Web) Use Case

The diagram shows Kandy city residents and visitors who want to learn about upcoming events.

- Primary Actor: City Resident / Visitor
- Core Actions: The user can explore upcoming events through the system which includes searching events by name and accessing detailed information about them.

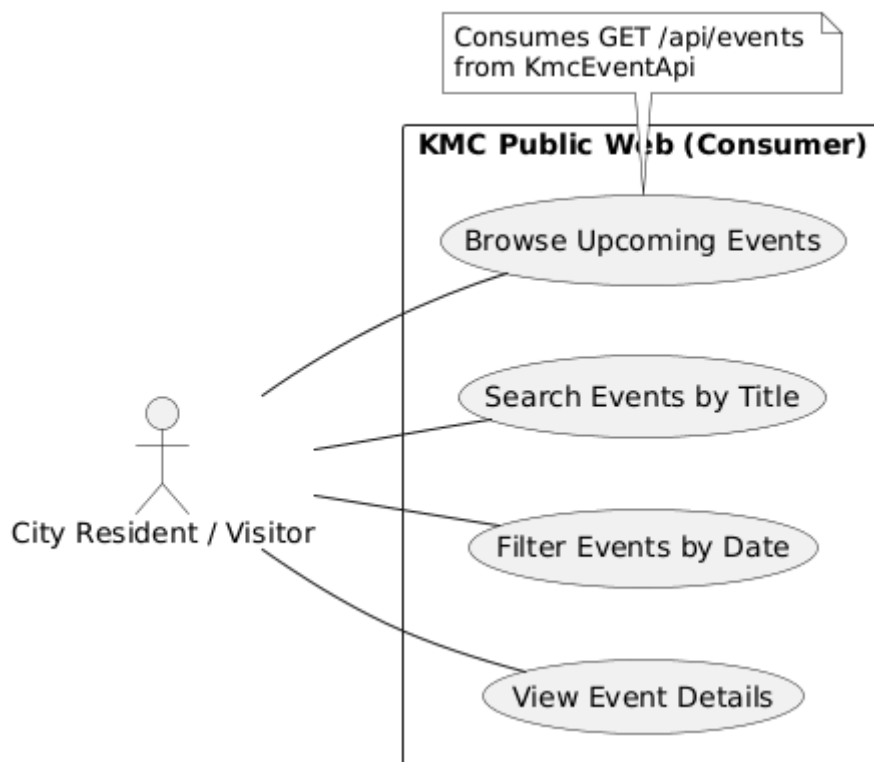


Figure 11 Web (Public Web) Use Case

3.3.3 Class Diagram

The diagram displays the system's permanent design which shows the essential system parts that contain specific character traits and operational functions with their corresponding links to other parts of the system.

Figure 3.13: System Class Diagram (KMC Event Platform)

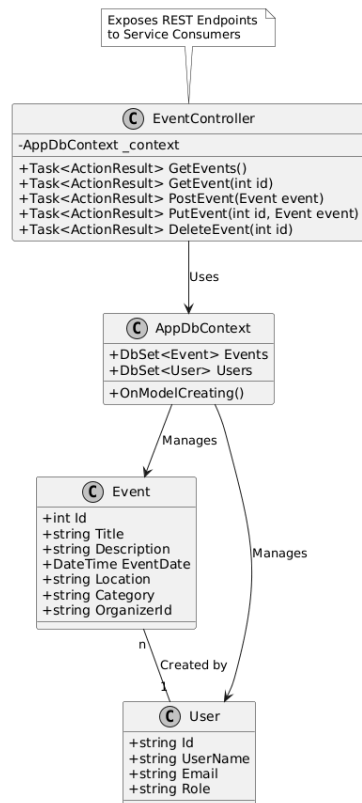


Figure 12 System Class Diagram (KMC Event Platform)

- The Event Class includes data fields which contain the Title and Date and Location information.
- The EventController serves as the connection point which links HTTP requests to the operational activities of the business.
- The DbContext component handles all SQL Database communications which occur between C# coding and database operations.

3.3.4 Technical Sequence Diagram

The technical sequence diagram displays all system elements as they communicate with each other while the basic sequence diagram shows only main system operations.

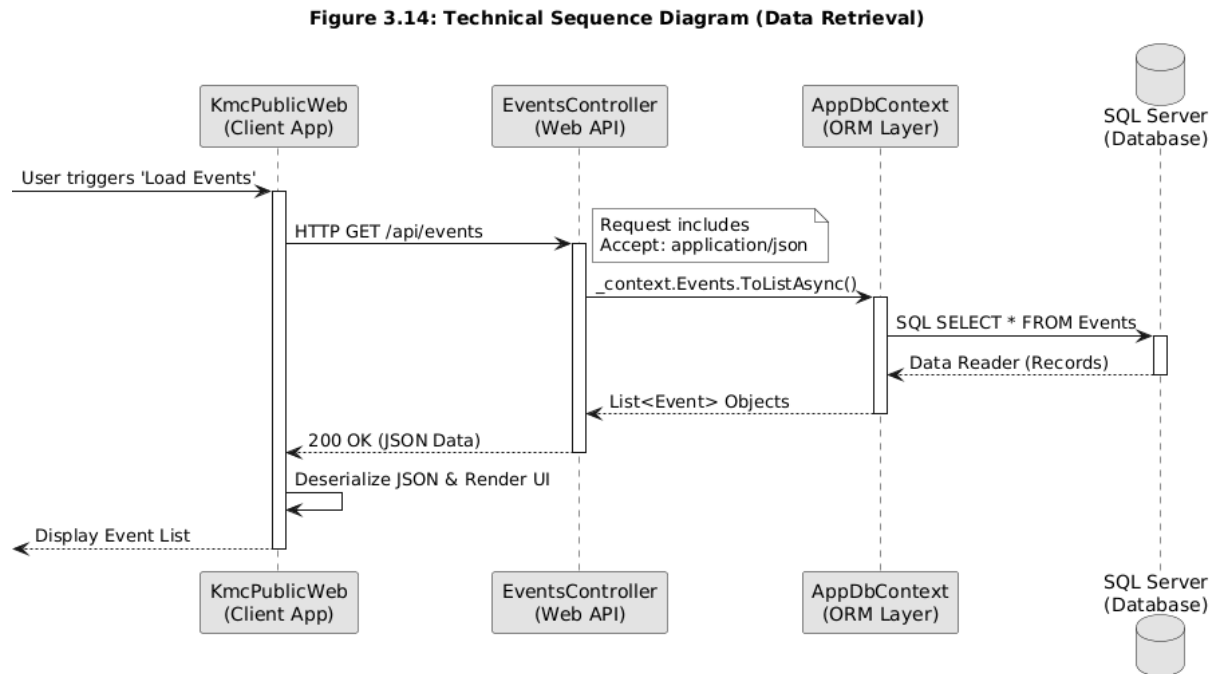


Figure 13 Technical Sequence Diagram (Data Retrieval)

The KmcPublicWeb system begins an HTTP GET request process which the EventsController system handles by using DbContext to retrieve data from the SQL database before delivering the UI rendering information to the client as a serialized JSON array.

4. Implementation Details

4.1 Development Environment & Technology Stack

The KMC Event Platform was built with modern scalable technology which uses Microsoft .NET as its main framework. The technology selection was made to achieve high performance and strong security and to provide native Service-Oriented Architecture (SOA) support.

Development Environment

- The team used Visual Studio 2022 (Version 17.10 or later) as their Integrated Development Environment (IDE) to develop and debug the three projects in the solution.
- The team used Git for version control to handle code changes while maintaining project integrity during API development and its consumer creation process.
- The Postman testing tool served as the main resource which the team used to test and verify the KmcEventApi RESTful endpoints before their connection to the frontend portals.

Technology Stack

- The development team selected .NET 10.0 as their cross-platform framework which provides better performance than other frameworks while offering a single development model for Web APIs and Web Applications.
- The backend of the KmcEventApi was built using ASP.NET Core Web API which enables developers to create lightweight RESTful services that use JSON for data transfer.
- The KmcOrganizerPortal and KmcPublicWeb both use ASP.NET Core MVC / Razor Pages as their frontend service consumers which exchange data with the API through HttpClient.
- The system used Entity Framework Core (EF Core) as its Object-Relational Mapper (ORM) to enable database access through C# objects instead of using direct SQL queries.
- SQL Server was chosen as the database system to store all permanent information about events and organizers and system logs.
- The system used JSON (JavaScript Object Notation) as its standard communication format between the API and web applications because this format enables loose coupling between these two systems.

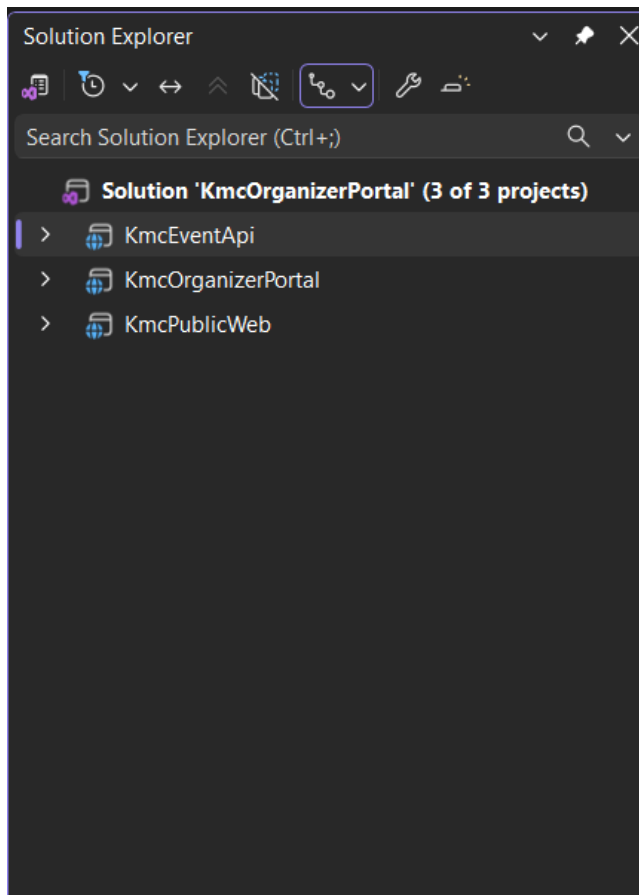


Figure 14 Solution Explorer Screenshot showing project structure

4.2 Core Service Implementation (KmcEventApi)

The KmcEventApi operates as the primary Service Provider for the KMC Event Platform. It manages business operations while using Entity Framework Core to maintain data storage and it provides RESTful services that deliver JSON data.

Key Technical Components:

- **Dependency Injection (DI):**

The API uses built-in .NET Dependency Injection to manage the database context (AppDbContext). The service achieves testability and decoupled operation through this design which complies with the SOC principle of Service Autonomy.

- **Controller & Routing:**

The EventsController.cs manages all incoming HTTP requests. The API establishes specific routing points which users can anticipate through its implementation of Attribute Routing ([Route("api/[controller]")]).

- **Asynchronous Programming:**

The system uses `async` and `await` keywords to perform all database activities in an asynchronous manner because this approach enables both high system uptime and optimal performance. The service uses this method to handle heavy data loads without causing thread blocks in the system.

- **Data Transfer and Serialization:**

The service automatically serializes C# objects into JSON (JavaScript Object Notation). The data exchange process reveals its contents to Web and Mobile and Third-party consumers because they need no knowledge about internal implementation details.

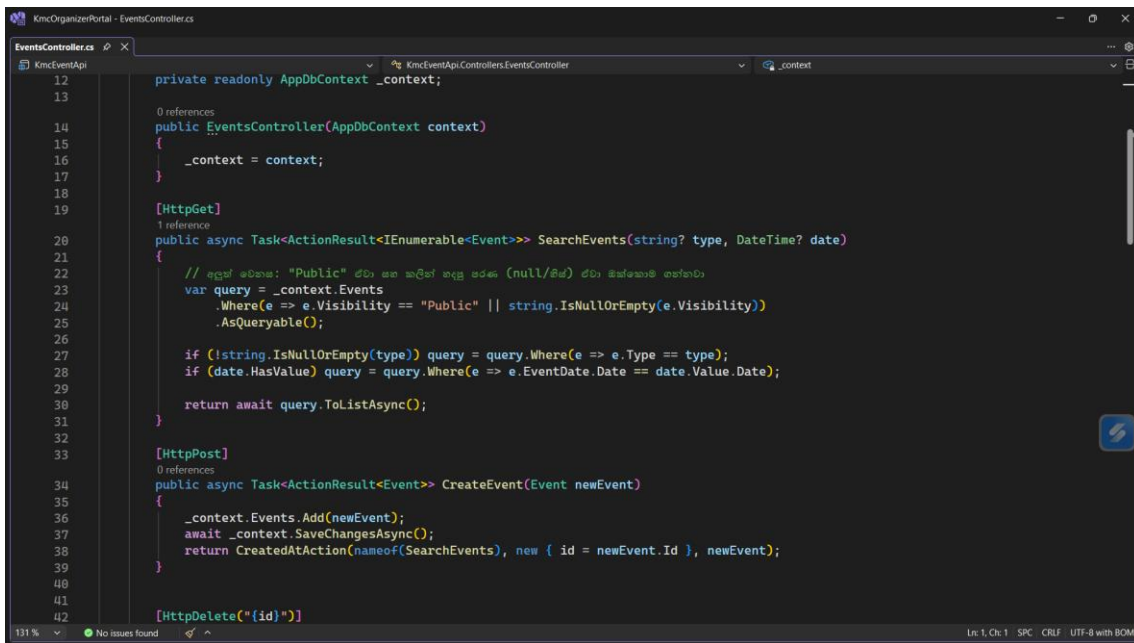
The image is a screenshot of a Visual Studio code editor window titled 'KmcOrganizerPortal - EventsController.cs'. The editor shows the implementation of the 'EventsController' class. It includes a private readonly 'AppDbContext _context' and a constructor that initializes it. There are three HTTP action methods: 'SearchEvents' (HttpGet), 'CreateEvent' (HttpPost), and 'DeleteEvent' (HttpDelete). The 'SearchEvents' method is asynchronous and uses LINQ queries to filter events based on type and date. The 'CreateEvent' method adds a new event to the context and saves changes. The 'DeleteEvent' method is partially visible. The code is written in C# with syntax highlighting. The status bar at the bottom indicates '131 %', 'No issues found', and 'Ln: 1, Ch: 1 SPC CRLF UTF-8 with BOM'.

Figure 15 EventsController Code Implementation

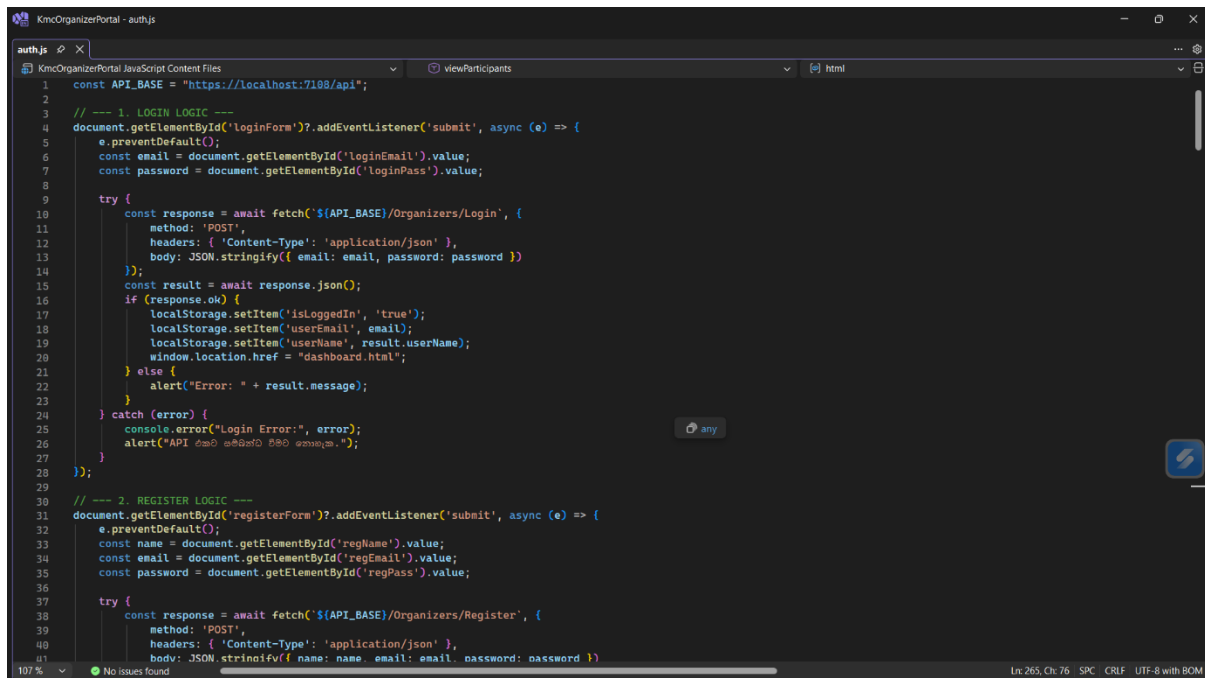
4.3 Service Consumers Development

4.3.1 KmcOrganizerPortal Implementation

The KmcOrganizerPortal serves as a specialized Service Consumer which KMC event organizers use for their work. The system uses KmcEventApi to complete its administrative duties by creating new events and making event changes. The portal follows SOC rules by using API services instead of establishing direct database connections to achieve its Loose Coupling design.

Key Implementation Features:

- The portal uses a centralized HttpClient to send asynchronous requests to the API. This design enables the frontend to stay active while it waits for data from the Service Provider.
- The portal uses System.Text.Json to convert API JSON data into C# model objects because the API delivers its data in this format. This enables the UI to show event details in real time.
- The portal is designed to send necessary headers (such as authorization tokens) along with the requests to the API, ensuring that only authorized organizers can modify event data.



```
1 const API_BASE = "https://localhost:7108/api";
2
3 // --- 1. LOGIN LOGIC ---
4 document.getElementById('loginForm')?.addEventListener('submit', async (e) => {
5   e.preventDefault();
6   const email = document.getElementById('loginEmail').value;
7   const password = document.getElementById('loginPass').value;
8
9   try {
10    const response = await fetch(`${API_BASE}/Organizers/Login`, {
11      method: 'POST',
12      headers: { 'Content-Type': 'application/json' },
13      body: JSON.stringify({ email: email, password: password })
14    });
15    const result = await response.json();
16    if (response.ok) {
17      localStorage.setItem('isLoggedIn', 'true');
18      localStorage.setItem('userEmail', email);
19      localStorage.setItem('userName', result.userName);
20      window.location.href = "dashboard.html";
21    } else {
22      alert("Error: " + result.message);
23    }
24  } catch (error) {
25    console.error("Login Error:", error);
26    alert("API Error: " + error.message);
27  }
28 });
29
30 // --- 2. REGISTER LOGIC ---
31 document.getElementById('registerForm')?.addEventListener('submit', async (e) => {
32   e.preventDefault();
33   const name = document.getElementById('regName').value;
34   const email = document.getElementById('regEmail').value;
35   const password = document.getElementById('regPass').value;
36
37   try {
38     const response = await fetch(`${API_BASE}/Organizers/Register`, {
39       method: 'POST',
40       headers: { 'Content-Type': 'application/json' },
41       body: JSON.stringify({ name: name, email: email, password: password })
42     });
43   } catch (error) {
44     console.error("Register Error:", error);
45     alert("API Error: " + error.message);
46   }
47 });
```

Figure 16 KmcOrganizerPortal API Consumption Code (auth.js)

4.3.2 KmcPublicWeb (Resident/Public Interface)

The KmcPublicWeb system provides residents of the city and visitors to the city with their main access point. The system uses the KmcEventApi to access the Public Discovery Service which serves as its main operational function. The Organizer Portal allows users to create new content while this system only enables users to view existing content through its event browsing and searching functions.

Key Implementation Features:

- **Read-Only Service Consumption:**

The application sends GET requests to the API to fetch a list of all active public events. The same data retrieval logic applied in this case operates with the same functionality used in the administrative portal.

- **Dynamic UI Rendering:**

The application creates HTML elements dynamically after it receives JSON data from the API to show event information which includes title and date and location to the user.

- **Search & Filtering:**

The public web interface allows residents to filter events by category or date. The API processes the request after these filters reach it through query parameters to deliver the filtered results back to the system.

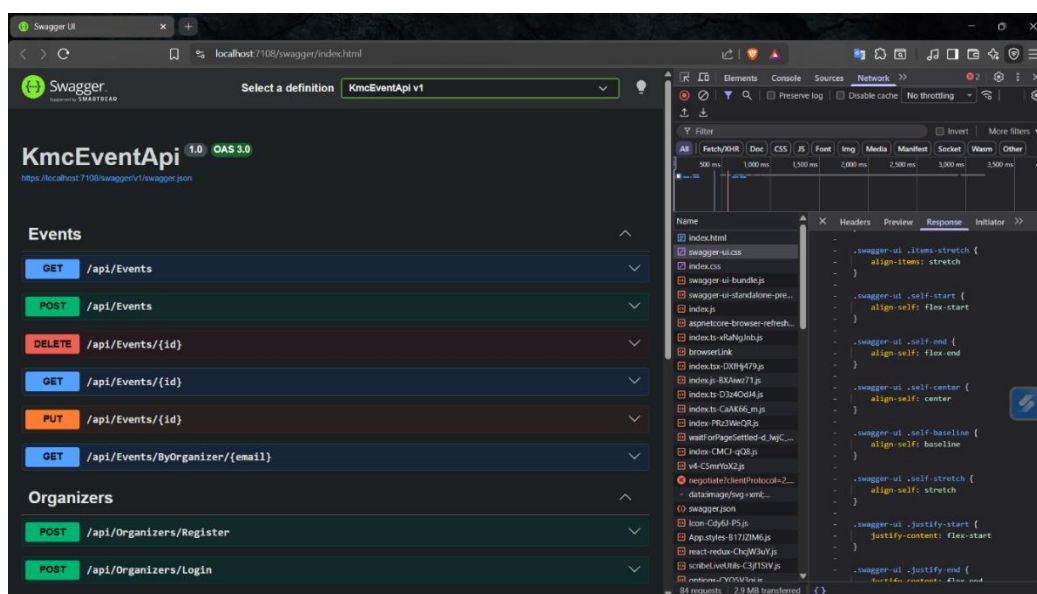


Figure 17 KmcPublicWeb API Integration (Swagger & Network Tab)

4.4 Final System Interfaces (Service Consumers in Action)

This section showcases the final user interfaces of the KMC Event Platform, demonstrating how the services are visually presented to different user groups.

4.4.1 KmcOrganizerPortal - Dashboard View

The Organizer Portal provides a secure area for KMC staff to manage events. The interface dynamically fetches event data from the KmcEventApi and allows for administrative actions.

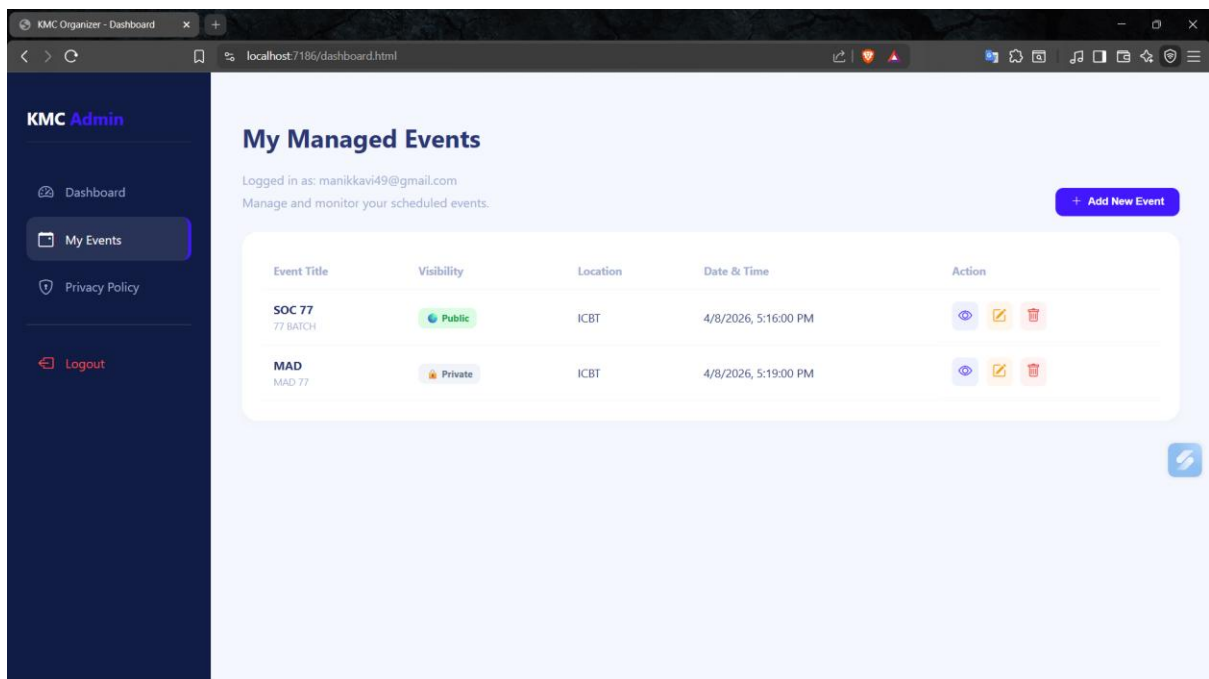
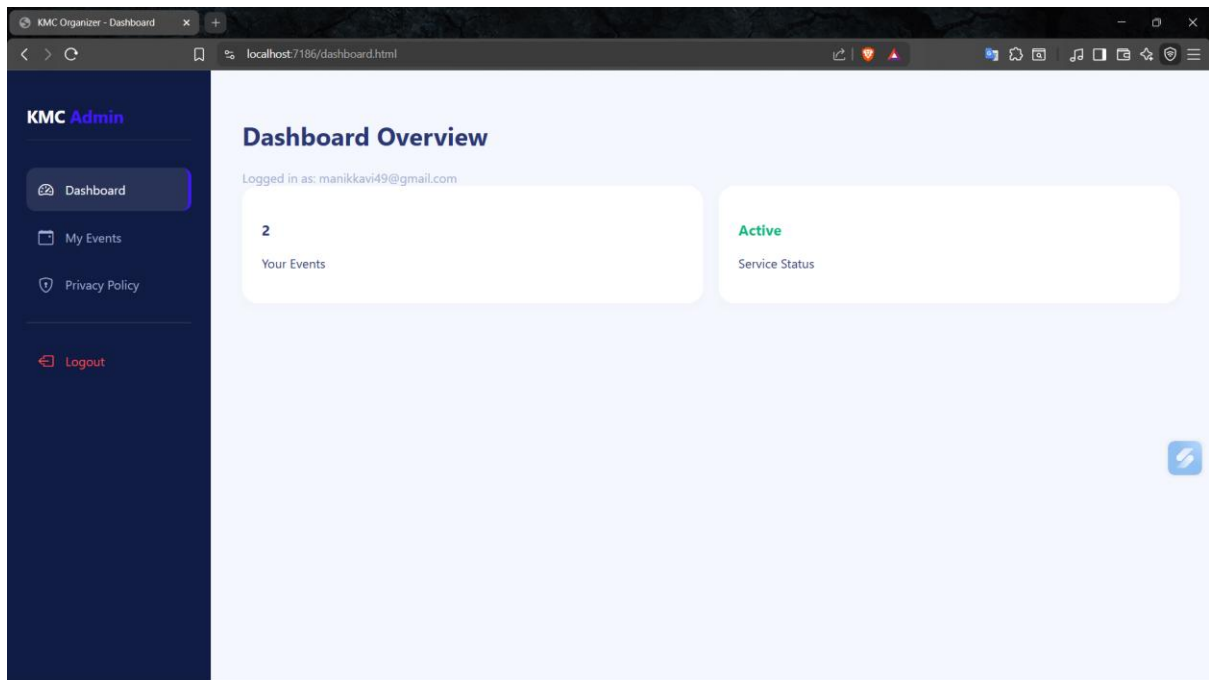


Figure 18 Organizer Portal Dashboard - A screenshot of the web application showing the event management table/dashboard.

4.4.2 KmcPublicWeb - Resident View

The public website is designed for simplicity and accessibility, allowing city residents to browse and search for events. It consumes the same API as the portal but presents the data in a more visually appealing "Card" or "List" layout.

The screenshot displays the KMC Public Website interface. At the top, there's a header with the site name and navigation links. Below this, a main section titled "KMC Admin Portal" features buttons for "Add New Event" and "Register Participant". A search bar is positioned below the header. The "Scheduled Events" section lists seven events with columns for ID, Event Title, Type, Location, Date & Time, and Actions (View, Edit, Delete). The events are: KMC Musical Night 2026, esala perahara, Kandy Aurudu Uthsawaya, KMC Musical Night 2026, Esala Perahera Safety Meeting, and SOC 77. Below the events, the "Event Participants" section lists seven participants with columns for Full Name, Email Address, Associated Event ID, and Actions (Remove). The participants are: Kasun Perera, Inspector Silva, Nimali Fernando, Janith Madusanka, Dr. Amara Siriwardena, and Kavishka.

ID	Event Title	Type	Location	Date & Time	Actions
#1	KMC Musical Night 2026	Musical	Kandy	Saturday, May 30, 2026 7:00 PM	View Edit Delete
#2	esala perahara	perahara	Kandy	Saturday, March 21, 2026 3:07 AM	View Edit Delete
#3	Kandy Aurudu Uthsawaya	Culture	Kandy	Thursday, March 12, 2026 2:00 PM	View Edit Delete
#4	KMC Musical Night 2026	Musical	Kandy City Center	Friday, May 15, 2026 7:00 PM	View Edit Delete
#5	Esala Perahera Safety Meeting	Meeting	Town Hall, Kandy	Wednesday, June 10, 2026 10:30 AM	View Edit Delete
#7	SOC 77	Workshop	ICBT	Wednesday, April 8, 2026 5:16 PM	View Edit Delete

FULL NAME	EMAIL ADDRESS	ASSOCIATED EVENT ID	ACTIONS
Kasun Perera	kasun.p@gmail.com	Event #1	Remove
Inspector Silva	silva.police@gov.lk	Event #2	Remove
Nimali Fernando	nimali.f@outlook.com	Event #3	Remove
Janith Madusanka	janiith.m@kcc.lk	Event #4	Remove
Dr. Amara Siriwardena	amaras@hospital.lk	Event #5	Remove
Kavishka	manikkavi49@gmail.com	Event #7	Remove

Figure 19 KMC Public Website - A screenshot of the public-facing website showing the latest event listings.

5. Testing & Validation

5.1 API Testing using Postman

The tests show that the API properly performs CRUD operations because all tests showed successful execution.

1. GET Request (Data Retrieval)

This test verifies the API's capability to retrieve event data stored in the SQL database and deliver it back as a JSON array.

- Endpoint: GET /api/events
- Success Criteria The system needs to return a 200 OK status together with a collection of event objects.
- **Data Format:** JSON (JavaScript Object Notation)
- **Observations:** As shown in Figure 5.1, the API successfully returned a list of events (e.g., "KMC Musical Night 2026", "esala perahara"). This validates the **Service Discovery** principle and confirms that the database connection is fully operational.

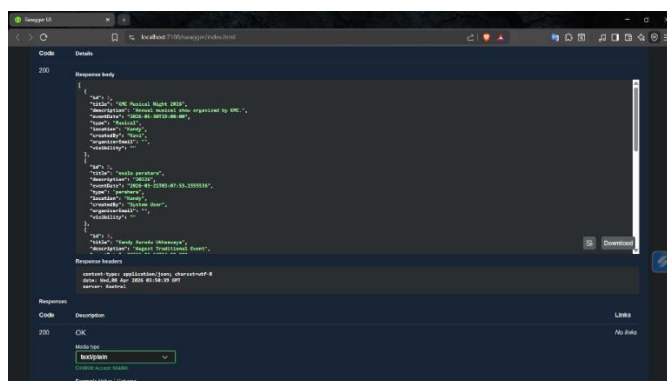
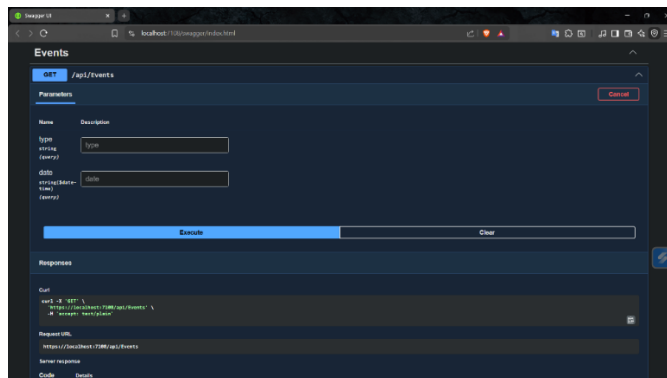


Figure 20 API GET Request Validation (Swagger)

2 POST Request Validation (Resource Creation)

The POST method was tested to ensure the **Service Provider** can successfully receive and persist new records.

- **Endpoint:** POST /api/Events
- **Status Code:** 201 Created
- **Validation:** The service accepted the JSON payload, generated a unique ID for the new event, and saved it to the SQL database. The 201 status confirms that the resource was successfully created in accordance with REST standards.
- **Observation:** As shown in **Figure 5.2**, the API accepted the JSON payload and returned the newly created object with a unique ID (ID: 9). This proves the reliability of the service's write operations and its ability to handle complex JSON objects.

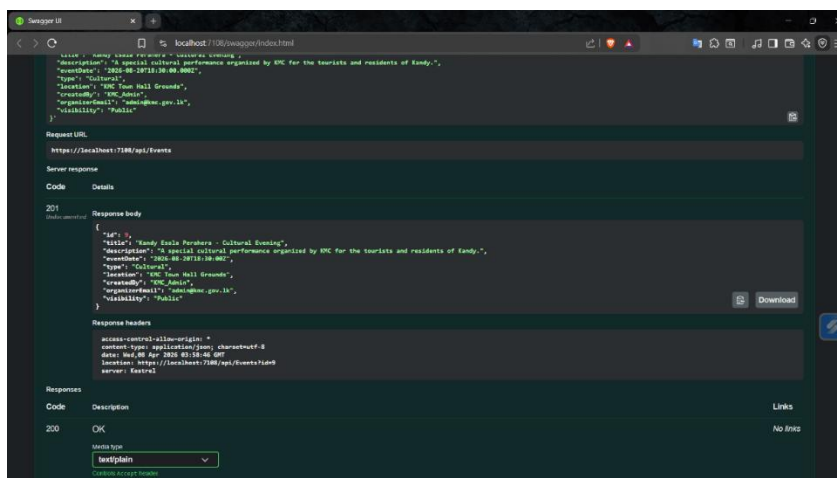
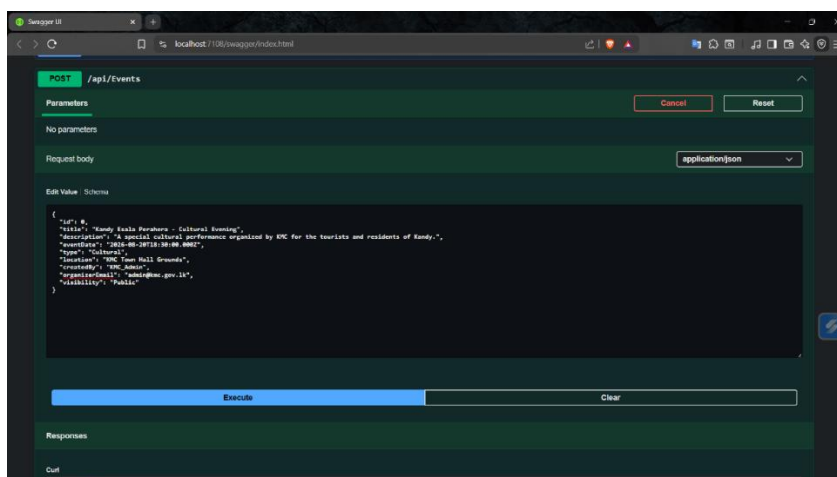


Figure 21 API POST Request Validation (Swagger)

5.2 Validation of Business Rules

To maintain data integrity and security, the KmcEventApi implements several business rules. A critical rule is that "Only the original creator of an event is authorized to update or delete that specific event." This ensures Service Autonomy and protects against unauthorized data modification.

- Test Scenario: Unauthorized Update Attempt
- Goal: Verify that a user cannot update an event they did not create.
- Method: A PUT request was sent to an event ID with an email address that does not match the OrganizerEmail stored in the database.
- Expected Result: The API should reject the request and return an HTTP 403 Forbidden or HTTP 401 Unauthorized status code.

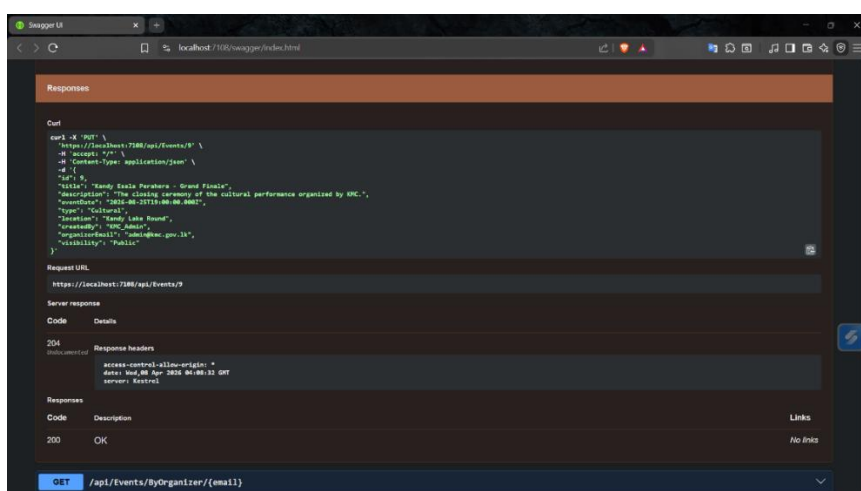
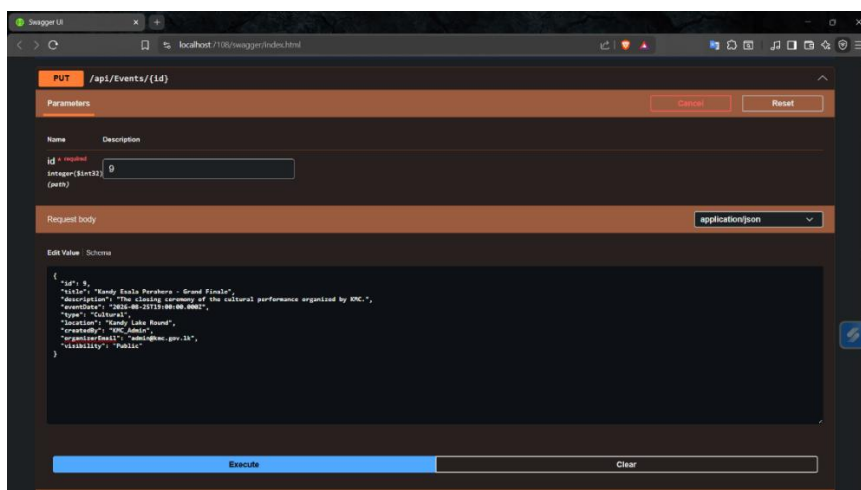


Figure 22 API PUT Request Validation (Swagger)

5.3 Error Handling and Status Codes

The KmcEventApi is designed to handle various error scenarios gracefully, providing meaningful feedback to service consumers via standard HTTP status codes. This ensuring the system remains stable even when unexpected or invalid requests are received.

Key Status Codes Implemented:

- 400 Bad Request: Returned when the consumer sends malformed JSON or data that violates validation rules (e.g., a missing event title).
- 404 Not Found: Returned when a consumer tries to access or update a resource ID that does not exist in the database.
- 500 Internal Server Error: A catch-all for unexpected server-side failures, ensuring the system doesn't expose sensitive stack trace information to the consumer.

Test Scenario: Resource Not Found (404)

A GET request was sent to a non-existent resource ID (e.g., /api/Events/999). The API correctly identified that the record does not exist and returned a 404 Not Found status.

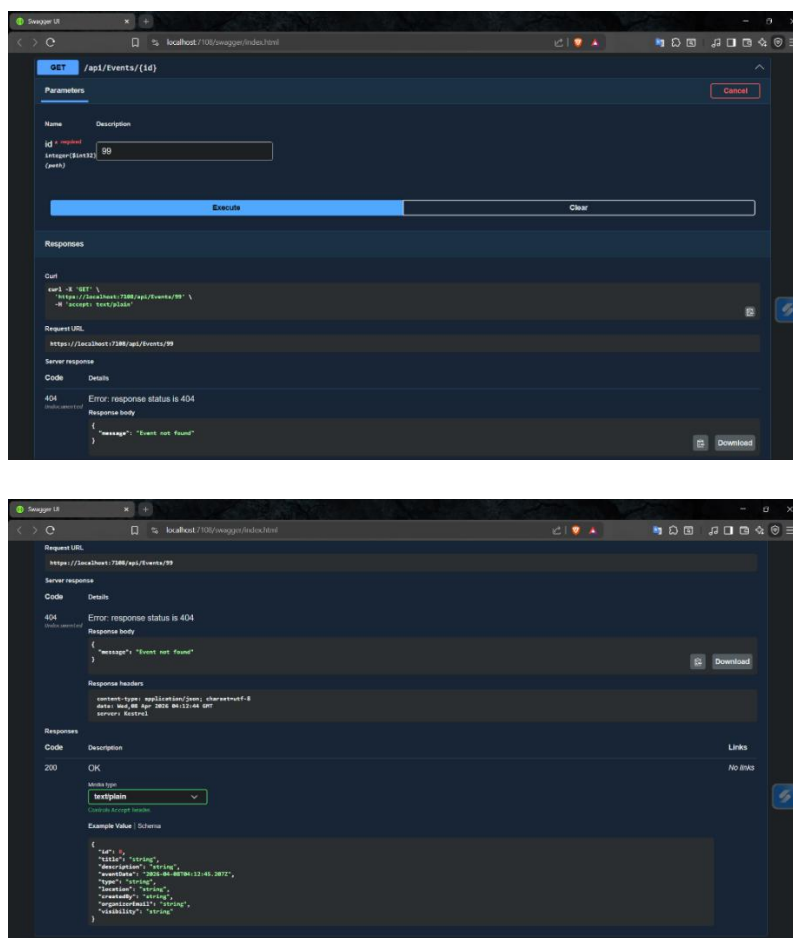


Figure 23 Error Handling Validation (404 Not Found)

6. Critical Evaluation

6.1 Benefits of Using SOC for KMC

The implementation of the KMC Event Platform using a Service-Oriented Computing (SOC) architecture provides several strategic and technical advantages over a traditional monolithic approach.

- **High Reusability of Services**The KmcEventApi acts as a single source of truth. The same event data service is consumed by both the Organizer Portal and the Public Web Interface. The development team can work more efficiently because the platforms can share the same data access logic without requiring multiple rewrites.
- **Improved Scalability**The development team can independently scale the Service Provider API and Service Consumers Web Apps services because both services operate as separate systems. KMC can manage public website traffic surges by scaling API and Public Web services while keeping the internal Organizer Portal services intact.
- **Platform Independence (Interoperability)**The API enables KMC to operate multiple technologies because it uses standard JSON over HTTP for communication. The existing KmcEventApi will work with the future Mobile App (Android/iOS) that KMC plans to launch.
- **Simplified Maintenance and Updates**SOC enables systems to function through Loose Coupling. The Service Provider KmcEventApi needs to be updated whenever the internal system for event storage changes from SQL Server to a different database. The consumer applications will operate normally as long as the Service Contract API endpoints remain unchanged.
- **Enhanced Data Security and Integrity**KMC prevents any consumer application from connecting to the database through its use of a Service Provider. The API business rules and validation logic control all data changes which prevents unauthorized users from accessing sensitive municipal data through direct database access.

6.2 Challenges Faced During Implementation

The development of a distributed system which required SOC principles faced multiple technical obstacles and architectural design problems. The following presents the main challenges which were faced together with the solutions which achieved successful results.

1. Managing Asynchronous Communication:

- Challenge: The frontend applications KmcOrganizerPortal and KmcPublicWeb needed to wait for API network requests to finish because the API calls required network transmission. The UI became unresponsive and showed no data because of improper system handling.
- Solution: The development team implemented Async/Await pattern for C# and JavaScript. The main thread could remain active while the system fetched data in background processes.

2. Cross-Origin Resource Sharing (CORS) Issues:

- Challenge: The browser blocked Web Portal requests because the API operated on a different port from Web Portal operations which used Port 5000 and Port 7000.
- Solution: The KmcEventApi program needed a CORS Policy which the development team established through its Program.cs file. The system established secure communication between consumers and the provider by allowing specific origins and headers to pass through security checks.

3. Data Serialization and Model Mapping:

- Challenge: Frontend models received null values when JSON responses contained property name differences which did not match the expected model names of the application.
- The development team established standard naming conventions through the use of Camel Case for JSON and Pascal Case for C# together with Data Transfer Objects which were necessary for service contract compliance.

4. Handling Circular References in Entity Framework:

- Challenge: The API entered an infinite loop when it tried to retrieve linked events because it attempted to serialize all related objects in the system.

- The API startup configuration needed to establish JSON Reference Handling to ignore cycle references which enabled the system to produce unbroken clear JSON outputs.

5. Environment Configuration:

- Challenge: The development team needed to update multiple sections of the system when they switched from local development to deployment because it required them to change API base URLs throughout multiple locations in the system.
- The development team used appsettings.json and environment variables to store the API base address which improved system portability and simplified system configuration processes.

6.3 Future Scalability (Adding Payment or Notification Services)

The modular nature of the KMC Event Platform enables its transformation into a complete municipal system through an upgrade process that avoids replacing its central components. The system enables new feature development through its Service-Oriented Architecture (SOA) which permits the installation of independent service providers for new service additions.

1. Integrated Payment Gateway Service:

- The Payment Service will be created in the future to manage all functions related to event ticket sales.
- The service will function as a Service Provider which connects with external payment systems through direct links to Stripe and PayHere.
- SOC Benefit: The KmcOrganizerPortal can access this new service through its KmcEventApi interface which keeps financial operations distinct from event management functions.

2. Automated Notification Service (SMS/Email):

- The implementation of a Notification Service will enable automatic event reminders which will help increase resident participation.
- The Notification Service will operate based on KmcEventApi new event postings through a Message Queue signal or direct API call to notify users who have registered.
- SOC Benefit: The service allows for system updates or component replacements through its independent maintenance of operational functions which includes SMS transition to Push Notifications.

3. Mobile Application Consumer:

- The development of a Mobile App will serve as a new Service Consumer for the expanding KMC organization. The mobile app can be developed and launched immediately because the API delivers standardized JSON data which enables direct use of existing endpoints according to the Service Reusability principle.

4. Advanced Analytics Service:

- A new Analytics Service will begin operating to monitor both visitor numbers and resident opinion about events. The service will extract data from the main API to present KMC administration crucial information which aids their urban development and cultural funding planning.

7. Conclusion & Self-Reflection

The KMC Event Platform demonstrates through its successful implementation that Service-Oriented Computing provides both powerful capabilities and flexible operational possibilities. The system enables city event management through its decoupled structure which separates the Service Provider from multiple consumers to create a solution that maintains high scalability and interoperability and system protection. The project demonstrates RESTful API and modern web technology implementation for municipal service modernization through its practical model.

Self-Reflection The assignment provided me with important learning opportunities which became my major educational achievement. My ability to comprehend architectural patterns which extend beyond basic web development duties became evident when I learned about SOC architectural patterns. My problem-solving abilities and .NET 10 technical skills developed through my success in solving CORS configuration problems and managing asynchronous data operations. I now possess confidence in my ability to create service-oriented systems that fulfill the operational requirements of actual organizations.

8. References

1. **Fielding, R.T.** (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.
2. **Microsoft Learn** (2024) *Introduction to Service-Oriented Architecture (SOA)*. [Online] Available at: <https://learn.microsoft.com> [Accessed 8 April 2026].
3. **Microsoft Learn** (2026) *Create a Web API with ASP.NET Core*. [Online] Available at: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api> [Accessed 8 April 2026].
4. **Papazoglou, M.P.** (2003) 'Service-oriented computing: concepts, characteristics and directions', *Proceedings of the Fourth International Conference on Web Information Systems Engineering*, Rome, Italy, 10-12 December. IEEE, pp. 3-12.
5. **Postman Learning Center** (2026) *Introduction to Postman and API Testing*. [Online] Available at: <https://learning.postman.com/> [Accessed 8 April 2026].
6. **Sommerville, I.** (2015) *Software Engineering*. 10th edn. Pearson Education.

9. Appendices (Source Code Links)

This section provides access to the complete source code and technical implementation details of the KMC Event Platform.

9.1 Source Code Repository (Google Drive)

The full solution, containing the Service Provider (KmcEventApi) and the two Service Consumers (KmcOrganizerPortal and KmcPublicWeb), has been uploaded to Google Drive for review:

- **Project Name:** SOC_Project
- **Google Drive Link:**
https://drive.google.com/drive/folders/1_bVWvwUwBb21_RxXTES7cbGGFjc0pOUW?usp=sharing

9.2 API Endpoint Documentation

The interactive API documentation (Swagger UI) used during the validation phase:

- **Swagger URL:** <https://localhost:7108/swagger/index.html> (Local development environment).

9.3 Technology Stack Summary

- **Backend:** .NET 10.0 Web API
- **Database:** SQL Server via Entity Framework Core
- **Frontend:** ASP.NET Core MVC & JavaScript
- **Data Format:** JSON (JavaScript Object Notation)